

▪ Comparison Table :

Feature	StatelessWidget	StatefulWidget
Changes Over Time	No	Yes
State Management	No internal state	Has internal state
Performance	More efficient	Less efficient
Example	Static text, icons, images	Interactive buttons, forms
Widget Class	class MyWidget extends StatelessWidget	class MyWidget extends StatefulWidget
Build Method	Only needs build method	Needs createState and State class

f. Comparison with Java (Android Development)

- In Java (Android), UI components are updated using View and Listener.

```
TextView textView = findViewById(R.id.myTextView);
textView.setText("Welcome to My App");
```

```
button.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        button.setText("You pressed the button!");
    }
});
```

- In Flutter, UI updates are handled declaratively using setState().

Understanding setState in Flutter

a. What is setState()?

- setState notifies Flutter that something has changed in the widget, triggering a UI rebuild.
- It takes a function where you modify the widget's state.
- Commonly used for handling user interactions like button clicks, form input, and animations.
- This is an example counter application :

```
class CounterWidget extends StatefulWidget {
    @override
    _CounterWidgetState createState() =>
    _CounterWidgetState();
}

class _CounterWidgetState extends State<CounterWidget> {
    int counter = 0;

    void incrementCounter() {
        setState(() {
            counter++; // Update the counter
        });
    }

    @override
```

```

Widget build(BuildContext context) {
  return Column(
    children: [
      Text('Counter: $counter'),
      ElevatedButton(
        onPressed: incrementCounter,
        child: Text('Increment'),
      ),
    ],
  );
}

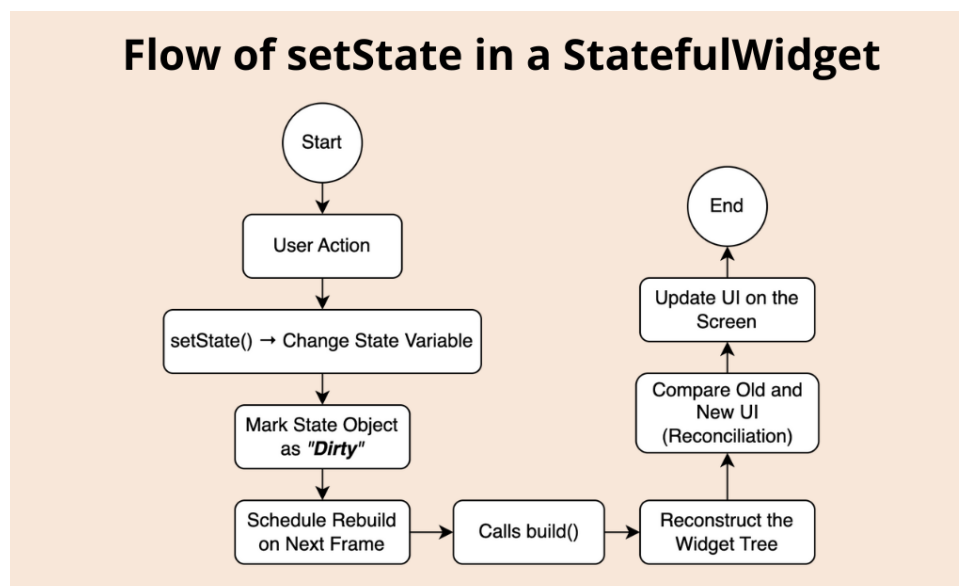
```

- Explanation:
 - Initially, counter is 0.
 - When the button is pressed, incrementCounter() is called.
 - Inside setState(), counter is updated, triggering a UI rebuild.
 - The new counter value is displayed on the screen.
- b. setState vs. Java (Android Development)

- Comparison Table

Feature	Flutter (setState)	Java (Android)
State Update Method	setState()	Directly modifying UI elements in event listeners
UI Update	Automatic	Manual
Rebuild Efficiency	Only update necessary parts	Requires explicit view updates
Example	Button changes text dynamically	Button modifies TextView manually

- Key Differences:
 - Java requires manually updating TextView.
 - Flutter's setState() automatically triggers a UI update.
 - Flutter uses a declarative approach, while Java follows an imperative model.
- c. Flow of setState()



1. State Change Triggered
 - When program call `setState(() { ... })`, the program change a value in the widget's state.
2. Mark Widget as "Dirty"
 - Flutter marks the State object as "dirty", meaning it needs to be rebuilt. This tells Flutter that something in the widget's state has changed, and the UI must be updated.
3. Schedule a Rebuild
 - Flutter schedules a rebuild of the widget. It doesn't update the screen immediately but waits until the next frame to process all UI updates efficiently.
4. `build()` Method is Called
 - The `build()` method in the State object is called again. Flutter reconstructs the widget tree only for the parts that depend on the changed state.
5. UI is Updated
 - Flutter compares the old widget tree and the new widget tree (a process called widget reconciliation). Only the parts of the UI that have changed are updated on the screen.

Summary of Stateless vs Stateful Widget

- StatelessWidget is for static UI, StatefulWidget is for interactive UI.
- Stateful widgets require a State class and `setState()` to update the UI.
- Flutter handles state declaratively, unlike Java's manual View updates.
- `setState()` is used to update StatefulWidgets and trigger UI rebuilds.
- It ensures Flutter efficiently updates only the necessary parts of the UI.
- Unlike Java (Android), Flutter handles state updates declaratively, avoiding manual UI modifications.
- The `setState()` process follows a structured flow: State Change → Mark Dirty → Schedule Rebuild → `build()` Called → UI Updated.